



PRECISE
Testing Solution

Web Application Security Testing Report

Report Of: **Tamil Nadu Backward Classes Commission**

Report Date: 10/09/2025

www.precisetestingsolution.com

A STQC And Cert-In Empanelled Agency

+91-8447088848



info@precisetestingsolution.com



G-256, Second Floor, Sector-63, Noida

Precise Testing Solution Pvt. Ltd (PTS)

Precise Testing Solution Pvt. Ltd is a private limited company based in Noida, Uttar Pradesh, India.

Precise Testing solution is working towards becoming a leading quality services provider for Testing and QC/QA for all types of domains and applications.

Introduction

PTS is committed to deliver best-in-class quality control assurance and software testing solutions to our clients. Using proven testing methodologies and industry's best practices. We help companies to optimize the quality, performance and availability of their applications. The wide range of testing solutions that we offer, give organizations a fast start in developing and deploying software of the highest quality while reducing the risks, costs and time associated with it.

PTS is **ISO 9001:2008** and **27001:2013** Certified and has already served **400+** customers and having completed **40,000** hours of testing. We have discovered more than **2,40,000** Vulnerabilities in various applications.

We are **STQC government of India** and **Cert-in** Empanelled Agency. We are only company in India which is Empanelled by two government agencies. We have certified, specialized and experienced testers for all types of testing. We have close to 100% project success ratio and 60% returned customer.

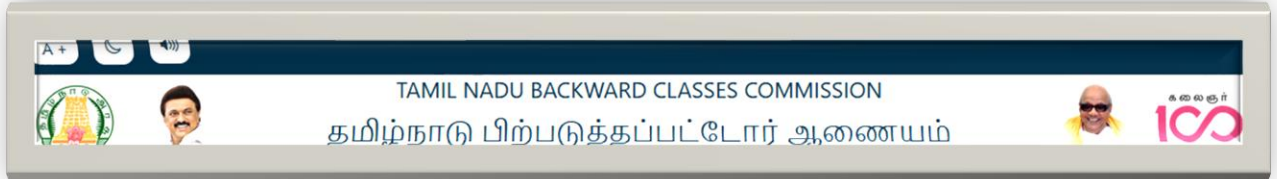
Our focused customers are Government and Defence

Document Version: 2025-2026

Report Ledger No	2025-001
Report Title	Web Application Security Testing Report
Report Submit Date	10/09/2025
Report Reference Code	PT-2025-WEB-09-08-TNC
Report Submit To:	Tamil Nadu Backward Classes Commission
Report Testing URL:	http://192.168.5.247/tnbcc/
Report Created By:	Precise Testing Solution Pvt. Ltd.
First Report	10-09-2025
1 st Revalidation Date	N/a
2 nd Revalidation Date	N/a
3 rd Revalidation Date	N/a
Final Closure Date	N/a

WEB APPLICATION SECURITY TESTING REPORT

Report For: Tamil Nadu Backward Classes Commission



Testing URL For Portal: <http://192.168.5.247/tnbcc/>

From

Precise Testing Solution Private Limited



Report Content: We have get Web Application and we have tested Web application security testing, testing perform via open internet.

During testing of web Application, we have try to covered all type of testing like Security testing and pen testing. All found Vulnerabilitys are mention below in Report.

Please find the below Web Application Security Testing standard as per **OWASP TOP 10 (2021)**

The Open Web Application Security Project (OWASP) is a non-profit foundation dedicated to improving the security of software. OWASP operates under an 'open community' model, where anyone can participate in and contribute to projects, events, online chats, and more. A guiding principle of OWASP is that all materials and information are free and easily accessed on their website, for everyone. OWASP offers everything from tools, videos, forums, projects, to events. In short, OWASP is a repository of all things web-application-security, backed by the extensive knowledge and experience of its open community contributors

A01:2021-Broken Access Control moves up from the fifth position to the category with the most serious web application security risk; the contributed data indicates that on average, 3.81% of applications tested had one or more Common Weakness Enumerations (CWEs) with more than 318k occurrences of CWEs in this risk category. The 34 CWEs mapped to Broken Access Control had more occurrences in applications than any other category.

A02:2021-Cryptographic Failures shifts up one position to #2, previously known as A3:2017-Sensitive Data Exposure, which was broad symptom rather than a root cause. The renewed name focuses on failures related to cryptography as it has been implicitly before. This category often leads to sensitive data exposure or system compromise.

A03:2021-Injection slides down to the third position. 94% of the applications were tested for some form of injection with a max incidence rate of 19%, an average incidence rate of 3.37%, and the 33 CWEs mapped into this category have the second most occurrences in applications with 274k occurrences. Cross-site Scripting is now part of this category in this edition.

A04:2021-Insecure Design is a new category for 2021, with a focus on risks related to design flaws. If we genuinely want to "move left" as an industry, we need more threat modeling, secure design patterns and principles, and reference architectures. An insecure design cannot be fixed by a perfect implementation as by definition, needed security controls were never created to defend against specific attacks.

A05:2021-Security Misconfiguration moves up from #6 in the previous edition; 90% of applications were tested for some form of Misconfiguration, with an average incidence rate of 4.5%, and over 208k occurrences of CWEs mapped to this risk category. With more shifts into

highly configurable software, it's not surprising to see this category move up. The former category for A4:2017-XML External Entities (XXE) is now part of this risk category.

A06:2021-Vulnerable and Outdated Components was previously titled Using Components with Known Vulnerabilities and is #2 in the Top 10 community survey, but also had enough data to make the Top 10 via data analysis. This category moves up from #9 in 2017 and is a known issue that we struggle to test and assess risk. It is the only category not to have any Common Vulnerability and Exposures (CVEs) mapped to the included CWEs, so a default exploit and impact weights of 5.0 are factored into their scores.

A07:2021-Identification and Authentication Failures was previously Broken Authentication and is sliding down from the second position, and now includes CWEs that are more related to identification failures. This category is still an integral part of the Top 10, but the increased availability of standardized frameworks seems to be helping.

A08:2021-Software and Data Integrity Failures is a new category for 2021, focusing on making assumptions related to software updates, critical data, and CI/CD pipelines without verifying integrity. One of the highest weighted impacts from Common Vulnerability and Exposures/Common Vulnerability Scoring System (CVE / CVSS) data mapped to the 10 CWEs in this category. A8:2017-Insecure Deserialization is now a part of this larger category.

A09:2021-Security Logging and Monitoring Failures was previously A10:2017-Insufficient Logging & Monitoring and is added from the Top 10 community survey (#3), moving up from #10 previously. This category is expanded to include more types of failures, is challenging to test for, and isn't well represented in the CVE / CVSS data. However, failures in this category can directly impact visibility, incident alerting, and forensics.

A10:2021-Server-Side Request Forgery is added from the Top 10 community survey (#1). The data shows a relatively low incidence rate with above average testing coverage, along with above-average ratings for Exploit and Impact potential. This category represents the scenario where the security community members are telling us this is important, even though it's not illustrated in the data at this time.

Report Creation Summary

A vulnerability assessment report is created to document security flaws found during testing, providing detailed insights into potential risks. The report follows a structured format to ensure clarity and consistency in assessing vulnerabilities. It includes severity classification, a title aligning with OWASP standards, and a description explaining the issue. Reference IDs, such as CVSS scores or CWE classifications, help standardize the assessment. The report also specifies the tools used, affected areas, and the impact of exploitation. A workaround section provides remediation steps, while references add credibility. A Proof of Concept (POC) demonstrates the exploit, and the outcome summarizes the findings. This format ensures a systematic approach to vulnerability reporting with Below Titles

Report Format: - Title used for the Report as per our standard Format

Severity: Specify the severity level: Critical, High, Medium, or Low. Impact if exploited

Title: Bug ID Name of vulnerability or as per OWASP Standard Description

Description: Brief explanation of the vulnerability.

Reference Id(s): CVSS Score, OWASP Category, CWE ID, or other relevant standard rating.

Vulnerability Identified By: Tools used for testing, e.g., Burp Suite, OWASP ZAP, Nmap, etc.

Vulnerable Points: Specific areas, endpoints, or links where the vulnerability was found.

Impact: Explain how an attacker can exploit vulnerability & consequences on the application.

Workaround: Steps to mitigate or fix the issue, including best security practices.

References Link: External links or references supporting the vulnerability details.

Proof Of Concept (POC): Brief steps or an example showing how the vulnerability was exploited.

Outcome: Final result or identification of the vulnerability after testing.

Severity Levels

Issues in this report are grouped into severity levels based on the technical risk presented by each item. Severity levels are ranked Critical, High, Medium, Low based on perceived overall threat.

CRITICAL	HIGH	MEDIUM	LOW
The defect that results in the termination of the complete system or one or more component of the system and causes extensive corruption of the data. The failed function is unusable and there is no acceptable alternative method to achieve the required results then the severity will be stated as critical	The defect must be resolved as soon as possible because the defect is affecting the application or the product severely. The system cannot be used until the repair has been done. Exploitable issue that could result in server compromise, access to sensitive application functionality, or disclosure of confidential data. Successful exploitation may require either user-level access to system/application or exploits may not be publicly available.	The defect should be resolved in the normal course of development activities. It can wait until a new build or version is created. Issue is typically leveraged in conjunction with one or more security issues to compromise the server, application, data, or user accounts. This level is also used to classify failures in error handling, auditing, or logging.	The defect is an irritant which should be repaired, but repair can be deferred until after more serious defect have been fixed. Results from general information leakage about the application, network, server, user accounts, or is inconsistent with security best practices and Guidelines. INFO The defect is with Informational type with minimum impact as Negligible urgency and impact

Total Number of Severity Found in Report

CRITICAL	HIGH	MEDIUM	LOW	INFO	TOTAL
00	03	03	01	00	07

Please find the below index with detail description of Vulnerabilities, Executed as per Web Application OWASP TOP 10-2021.

OWASP TOP 10 2021	Vulnerability / Vulnerability Name	Result	Vulnerability ID Found:
A1	A01:2021-Broken Access Control	Not Found	N/a
A2	A02:2021-Cryptographic Failures	Not Found	N/a
A3	A03:2021-Injection	Found	Bug ID:01, Bug ID:02
A4	A04:2021-Insecure Design	Not Found	Bug ID:03
A5	A05:2021-Security Misconfiguration	Found	Bug ID:04, Bug ID:07
A6	A06:2021-Vulnerable and Outdated Components	Found	Bug ID:05, Bug ID:06
A7	A07:2021-Identification and Authentication Failures	Not Found	N/a
A8	A08:2021-Software and Data Integrity Failures	Not Found	N/a
A9	A09:2021-Security Logging and Monitoring Failures	Not Found	N/a
A10	A10:2021-Server-Side Request Forgery	Not Found	N/a

Reference:

<https://owasp.org/Top10/>

<https://owasp.org/www-project-top-ten/>

Vulnerability Table & Content

VULNERABILITY DESCRIPTION	SEVERITY	Status
BUG ID: 01 - Stored XSS	High	Open
BUG ID: 02 - HTML Injection	High	Open
BUG ID: 03 - Malicious File Upload	High	Open
BUG ID: 04 - Brower Cache Weakness	Medium	Open
BUG ID: 05 – Outdated JQuery Version	Medium	Open
BUG ID: 06 – Outdated Bootstrap Version	Medium	Open
BUG ID: 07 - Server Name Disclosure	Low	Open

Table of Contents

#PT-LDS-BUG ID 01: Stored XSS	11
#PT-LDS-BUG ID 02: HTML Injection	19
#PT-LDS-BUG ID 03: Malicious File Upload	27
SEVERITY: Medium.....	34
#PT-LDS-BUG ID 04: Browser Cache Weakness	34
#PT-LDS-BUG ID 05: Outdated JQuery.....	39
#PT-LDS-BUG ID 06: Outdated Bootstrap Version	43
SEVERITY: LOW.....	46
#PT-LDS-BUG ID 07: Server Name Disclosure	46

SEVERITY: HIGH#PT-LDS-BUG ID 01: **Stored XSS****Description**

Stored Cross-Site Scripting (XSS) is a type of security vulnerability commonly found in web applications. It occurs when an attacker injects malicious scripts (typically JavaScript) into a web application's database, which are then permanently stored and executed when other users access the affected page. Unlike reflected XSS, where the malicious script is reflected off a web server (e.g., in a URL or form submission), stored XSS persists on the server and impacts multiple users.

Reference Id(s):

OWASP TOP 10 - 2021	CVSS	CWE	WASC	Status	Severity
A03:2021-Injection	8.1	79	8	Open	High

Vulnerability Identified By:**How It Was Discovered**

☒ Manual
 ☐ Burp Suite
 ☐ Kali Linux
 ☐ SQLMap
 ☐ Other Tool

Vulnerable Points:

Affected URL / Sub-link)	Affected Link (Main)	User – Role / Impacted URL
http://192.168.5.247/tnbcc/admin-acts-rules.html	http://192.168.5.247/tnbcc/	User: ADMIN Pass: Admin@123 Role: admin
		http://192.168.5.247/tnbcc/admin-acts-rules.html

Impact

Implications / Consequences of not Fixing the Issue

1. Data Theft

Attackers can steal sensitive information from users, such as:

Session cookies (enabling session hijacking).

Login credentials.

Personal data (e.g., names, addresses, payment information).

This can lead to identity theft, financial fraud, or unauthorized access to user accounts.

2. Session Hijacking

By stealing session cookies, attackers can impersonate legitimate users and gain unauthorized access to their accounts.

This can result in account takeover (ATO) and unauthorized actions on behalf of the victim.

3. Defacement of Websites

Attackers can modify the content of web pages to display malicious or inappropriate content.

This can damage the reputation of the organization and erode user trust.

4. Malware Distribution

Attackers can use Stored XSS to deliver malware to users' devices by redirecting them to malicious websites or forcing downloads of malicious files.

This can lead to further compromise of user systems.

5. Phishing Attacks

Attackers can inject fake login forms or other phishing content into legitimate web pages.

Users may unknowingly provide sensitive information to attackers, thinking they are interacting with a trusted site.

6. Escalation to Other Vulnerabilities

Stored XSS can be combined with other vulnerabilities (e.g., CSRF, privilege escalation) to amplify the attack's impact.

For example, an attacker could use XSS to perform actions on behalf of an admin user, leading to full system compromise.

7. Reputation Damage

If users are affected by an XSS attack, they may lose trust in the application or organization. This can lead to a loss of customers, revenue, and brand credibility.

8. Regulatory and Legal Consequences

If sensitive user data is compromised, organizations may face legal penalties under data protection regulations (e.g., GDPR, CCPA).

Fines, lawsuits, and mandatory audits are possible outcomes.

9. Business Disruption

If the attack is widespread, it could disrupt normal business operations, requiring significant time and resources to remediate.

Workaround:

Solution /Mitigation / Remediation

1. Input Validation

Sanitize User Input: Ensure all user-supplied data is sanitized to remove or neutralize malicious content.

Whitelist Valid Input: Use allowlists to accept only expected and safe input formats (e.g., alphanumeric characters for usernames).

Reject Malicious Input: Block or escape special characters (e.g., <, >, &, ", ') that could be used in XSS payloads.

2. Output Encoding

Encode Dynamic Content: Always encode user-generated content before rendering it in the browser. Use context-specific encoding (e.g., HTML, JavaScript, URL) to prevent browsers from interpreting it as executable code.

Use Secure Libraries: Leverage libraries like OWASP's Java Encoder, DOMPurify, or HTMLSanitizer to automate output encoding.

3. Content Security Policy (CSP)

Implement CSP Headers: Use CSP to restrict the sources from which scripts can be executed. For

example:

Content-Security-Policy: default-src 'self'; script-src 'self' https://trusted.cdn.com;

Disable Inline Scripts: Prevent the execution of inline scripts and eval() functions.

Report Violations: Use the report-uri or report-to directive to log CSP violations for further analysis.

4. Use Secure Frameworks and Libraries

Leverage Modern Frameworks: Use frameworks like React, Angular, or Vue.js, which automatically handle XSS protections by default.

Avoid Risky Functions: Avoid using dangerous functions like innerHTML or document.write() in JavaScript.

5. HTTP-Only and Secure Cookies

Mark Cookies as HTTP-Only: Prevent client-side scripts from accessing sensitive cookies.

Use Secure Flag: Ensure cookies are only transmitted over HTTPS to prevent interception.

6. Regular Security Testing

Static and Dynamic Analysis: Use tools like OWASP ZAP, Burp Suite, or SonarQube to identify XSS vulnerabilities during development and testing.

Penetration Testing: Conduct regular penetration tests to uncover potential XSS vulnerabilities in production environments.

7. Educate Developers

Security Training: Train developers on secure coding practices and the risks of XSS.

Code Reviews: Conduct peer reviews to catch potential XSS vulnerabilities before code is deployed.

8. Use Web Application Firewalls (WAFs)

Deploy a WAF: Use a WAF to detect and block XSS payloads in real-time.

Regularly Update Rules: Ensure the WAF is configured with up-to-date rules to protect against emerging XSS techniques.

9. Escape Data in Different Contexts

HTML Context: Use HTML entity encoding (e.g., < becomes <).

JavaScript Context: Use JavaScript escaping (e.g., " becomes \x22).

URL Context: Use URL encoding (e.g., becomes %20).

10. Database Sanitization

Sanitize Data Before Storage: Ensure data stored in the database is sanitized to prevent malicious scripts from being retrieved and executed later.

References Link or URL if Any:

- <https://portswigger.net/web-security/cross-site-scripting/stored>

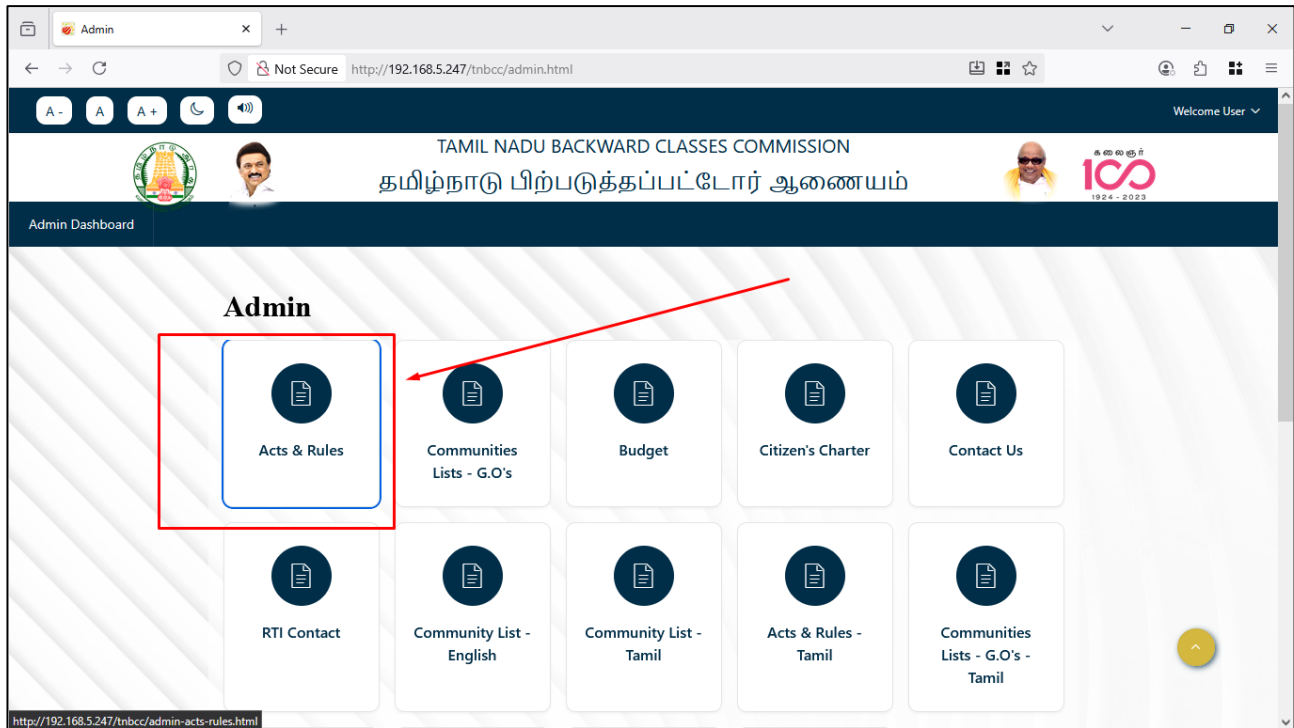
Proof Of Concept (POC)

Step 1:- Login with given credentials

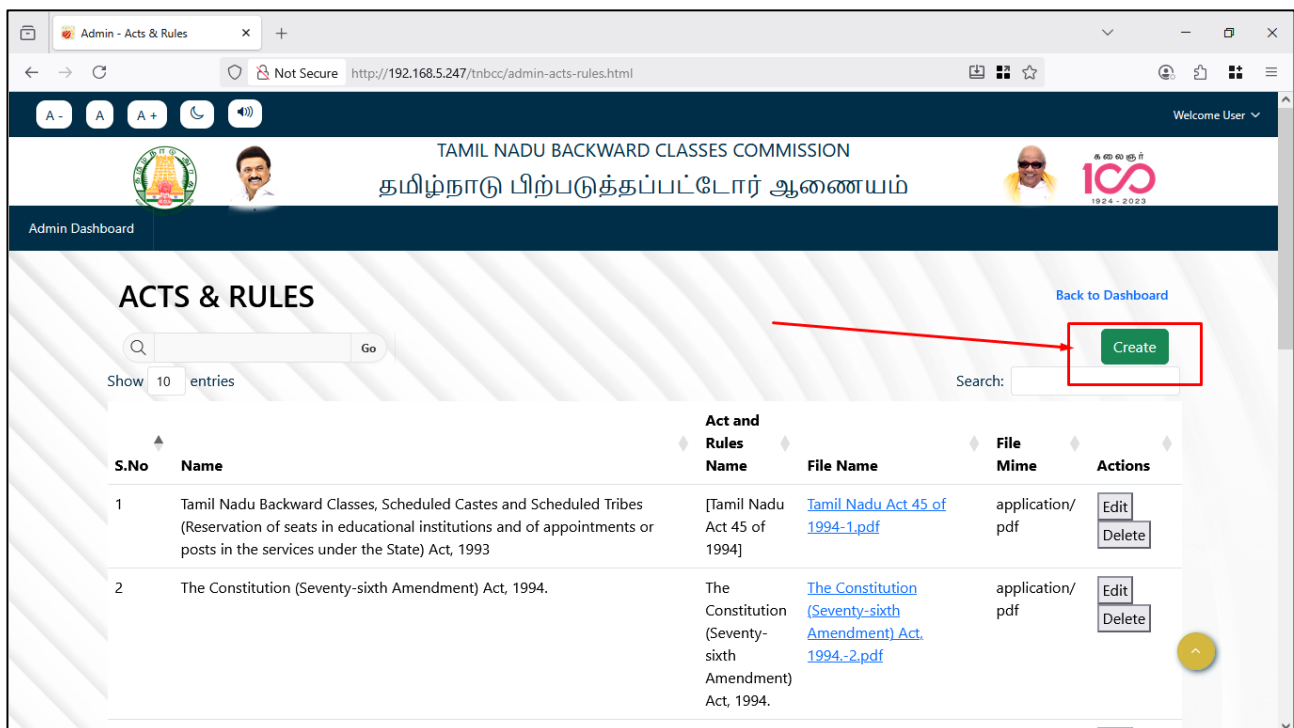
username = ADMIN

password = Admin@123

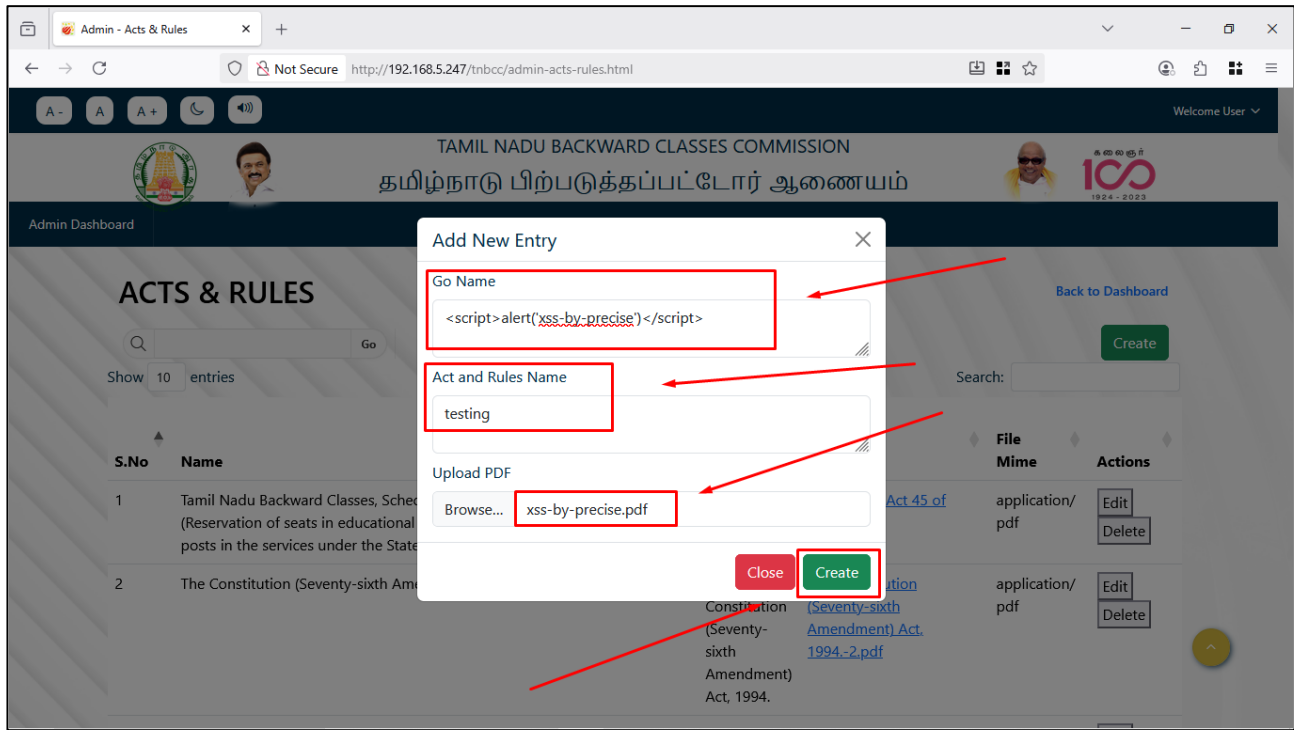
Step 2:- Click on *Acts & Rules*



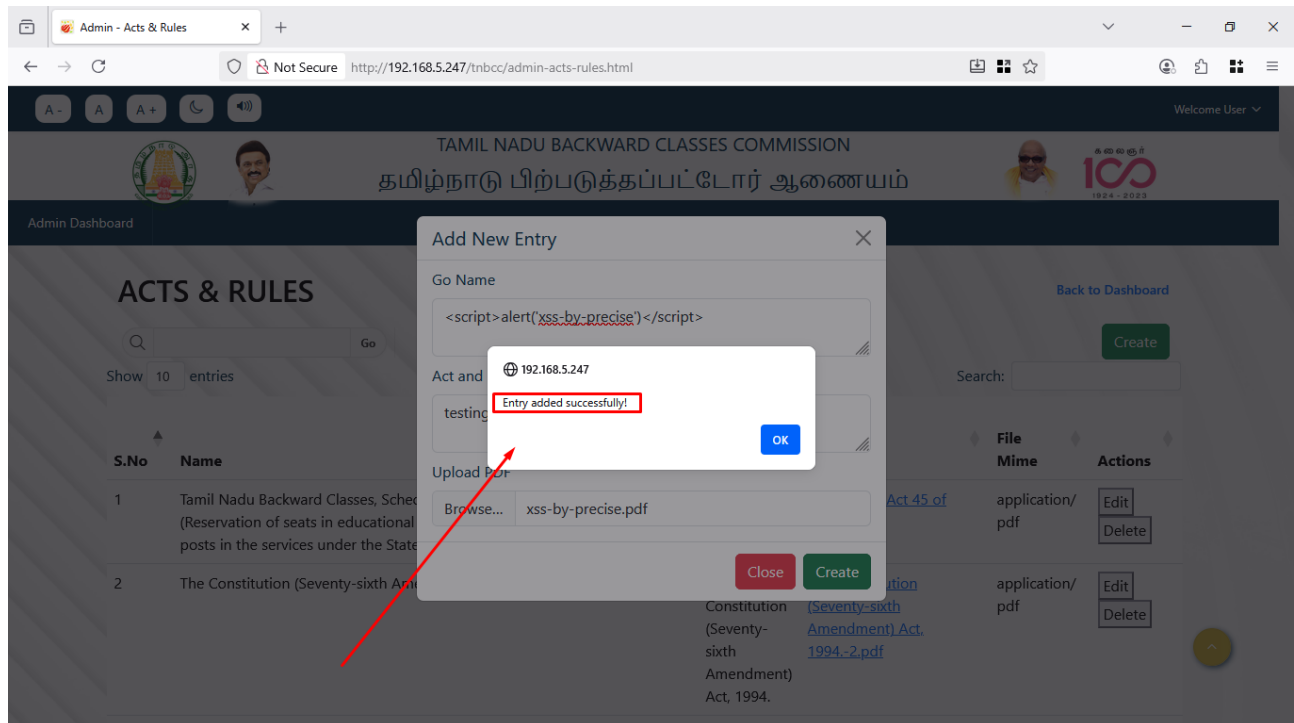
Step 3:- Click on create button

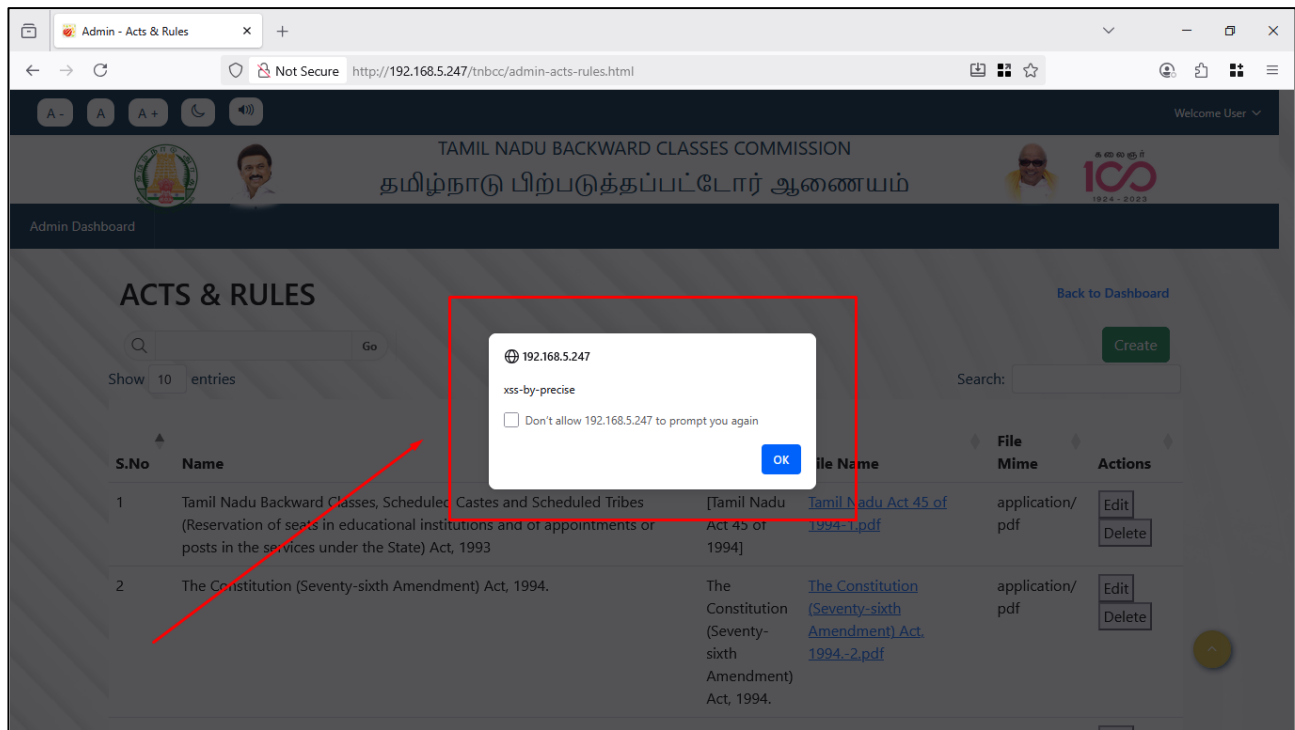


Step 4:- enter normal xss payload `<script>alert('xss-by-precise')</script>` in Go Name Field
fill the form and choose any pdf file then create new entry



Step 4:- Entry added successfully!





Outcome:

Stored XSS

SEVERITY: HIGH#PT-LDS-BUG ID 02: **HTML Injection****Description**

Stored HTML Injection is a type of security vulnerability where an attacker injects malicious HTML or JavaScript code into a web application's database. This injected content is then permanently stored and rendered to other users when they access the affected page. Unlike Reflected HTML Injection, which involves immediate execution of injected code, Stored HTML Injection persists on the server and impacts multiple users over time.

Reference Id(s):

OWASP TOP 10 - 2021	CVSS	CWE	WASC	Status	Severity
A03:2021-Injection	8.2	79	08	Open	High

Vulnerability Identified By:**How It Was Discovered**

☒ Manual
 ☐ Burp Suite
 ☐ Kali Linux
 ☐ SQLMap
 ☐ Other Tool

Vulnerable Points:

Affected URL / Sub-link)	Affected Link (Main)	User – Role / Impacted URL
http://192.168.5.247/tnbcc/admin-acts-rules.html	http://192.168.5.247/tnbcc/admin-acts-rules.html	User: ADMIN Role: admin Pass: Admin@123
		http://192.168.5.247/tnbcc/admin-acts-rules.html

Impact

Implications / Consequences of not Fixing the Issue

1. Data Theft

Attackers can steal sensitive information from users, such as:

Session cookies (enabling session hijacking).

Login credentials.

Personal data (e.g., names, addresses, payment information).

This can lead to identity theft, financial fraud, or unauthorized access to user accounts.

2. Session Hijacking

By stealing session cookies, attackers can impersonate legitimate users and gain unauthorized access to their accounts.

This can result in account takeover (ATO) and unauthorized actions on behalf of the victim.

3. Defacement of Websites

Attackers can modify the content of web pages to display malicious or inappropriate content.

This can damage the reputation of the organization and erode user trust.

4. Malware Distribution

Attackers can use Stored HTML Injection to deliver malware to users' devices by redirecting them to malicious websites or forcing downloads of malicious files.

This can lead to further compromise of user systems.

5. Phishing Attacks

Attackers can inject fake login forms or other phishing content into legitimate web pages.

Users may unknowingly provide sensitive information to attackers, thinking they are interacting with a trusted site.

6. Escalation to Other Vulnerabilities

Stored HTML Injection can be combined with other vulnerabilities (e.g., CSRF, privilege escalation) to amplify the attack's impact.

For example, an attacker could use HTML Injection to perform actions on behalf of an admin user, leading to full system compromise.

7. Reputation Damage

If users are affected by an HTML Injection attack, they may lose trust in the application or organization.

This can lead to a loss of customers, revenue, and brand credibility.

8. Regulatory and Legal Consequences

If sensitive user data is compromised, organizations may face legal penalties under data protection regulations (e.g., GDPR, CCPA).

Fines, lawsuits, and mandatory audits are possible outcomes.

9. Business Disruption

If the attack is widespread, it could disrupt normal business operations, requiring significant time and resources to remediate.

Workaround:

Solution /Mitigation / Remediation

1. Input Validation

Sanitize User Input: Ensure all user-supplied data is sanitized to remove or neutralize malicious content.

Whitelist Valid Input: Use allowlists to accept only expected and safe input formats (e.g., alphanumeric characters for usernames).

Reject Malicious Input: Block or escape special characters (e.g., <, >, &, ", ') that could be used in HTML Injection payloads.

2. Output Encoding

Encode Dynamic Content: Always encode user-generated content before rendering it in the browser. Use context-specific encoding (e.g., HTML, JavaScript, URL) to prevent browsers from interpreting it as executable code.

Use Secure Libraries: Leverage libraries like OWASP's Java Encoder, DOMPurify, or HTMLSanitizer to automate output encoding.

3. Content Security Policy (CSP)

Implement CSP Headers: Use CSP to restrict the sources from which scripts can be executed. For example

Content-Security-Policy: default-src 'self'; script-src 'self' https://trusted.cdn.com;

Disable Inline Scripts: Prevent the execution of inline scripts and eval() functions.

Report Violations: Use the report-uri or report-to directive to log CSP violations for further analysis.

4. Use Secure Frameworks and Libraries

Leverage Modern Frameworks: Use frameworks like React, Angular, or Vue.js, which automatically handle XSS protections by default.

Avoid Risky Functions: Avoid using dangerous functions like innerHTML or document.write() in JavaScript.

5. HTTP-Only and Secure Cookies

Mark Cookies as HTTP-Only: Prevent client-side scripts from accessing sensitive cookies.

Use Secure Flag: Ensure cookies are only transmitted over HTTPS to prevent interception.

6. Regular Security Testing

Static and Dynamic Analysis: Use tools like OWASP ZAP, Burp Suite, or SonarQube to identify HTML Injection vulnerabilities during development and testing.

Penetration Testing: Conduct regular penetration tests to uncover potential HTML Injection vulnerabilities in production environments.

7. Educate Developers

Security Training: Train developers on secure coding practices and the risks of HTML Injection.

Code Reviews: Conduct peer reviews to catch potential HTML Injection vulnerabilities before code is deployed.

8. Use Web Application Firewalls (WAFs)

Deploy a WAF: Use a WAF to detect and block HTML Injection payloads in real-time.

Regularly Update Rules: Ensure the WAF is configured with up-to-date rules to protect against emerging HTML Injection techniques.

9. Escape Data in Different Contexts

HTML Context: Use HTML entity encoding (e.g., < becomes <).

JavaScript Context: Use JavaScript escaping (e.g., " becomes \x22).

URL Context: Use URL encoding (e.g., becomes %20).

10. Database Sanitization

Sanitize Data Before Storage: Ensure data stored in the database is sanitized to prevent malicious scripts from being retrieved and executed later.

References Link or URL if Any:

- <https://hackerone.com/reports/1252155>

Proof Of Concept (POC)

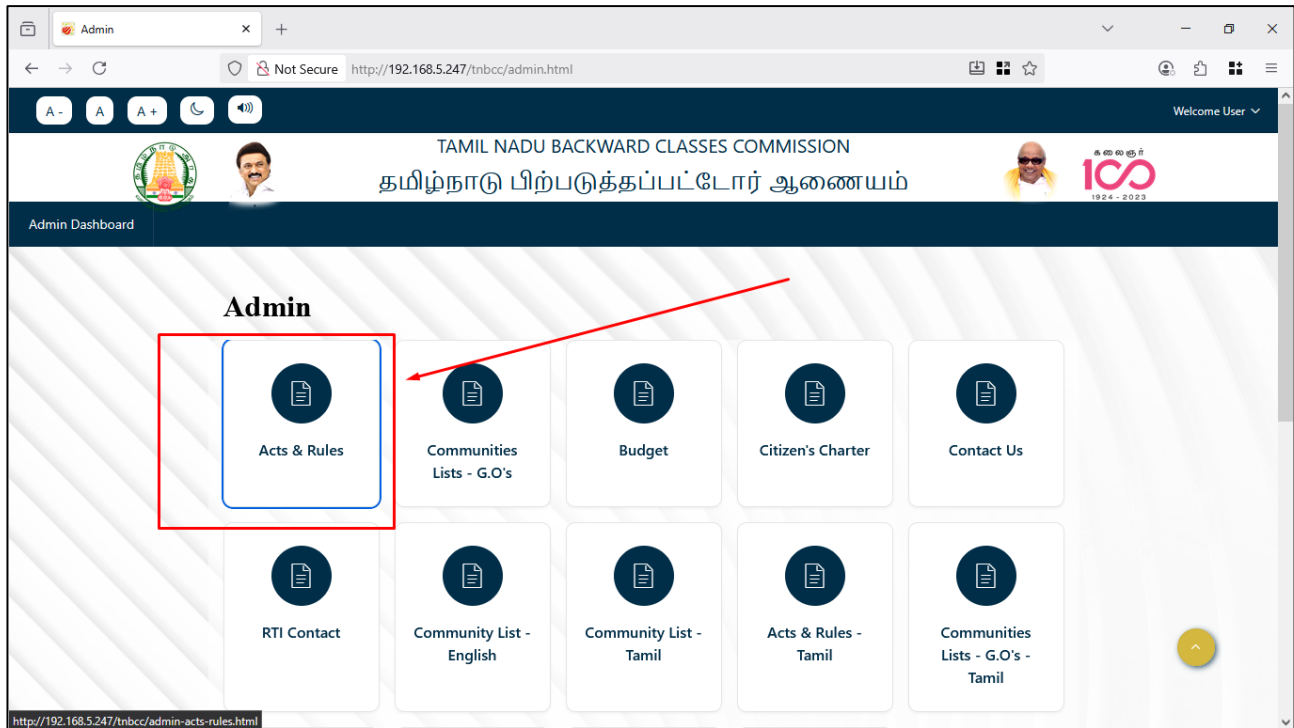
Step 1:- Access the web application URL – <http://192.168.5.247/tnbcc/admin-acts-rules.html>

Step 2:- Login with given credentials

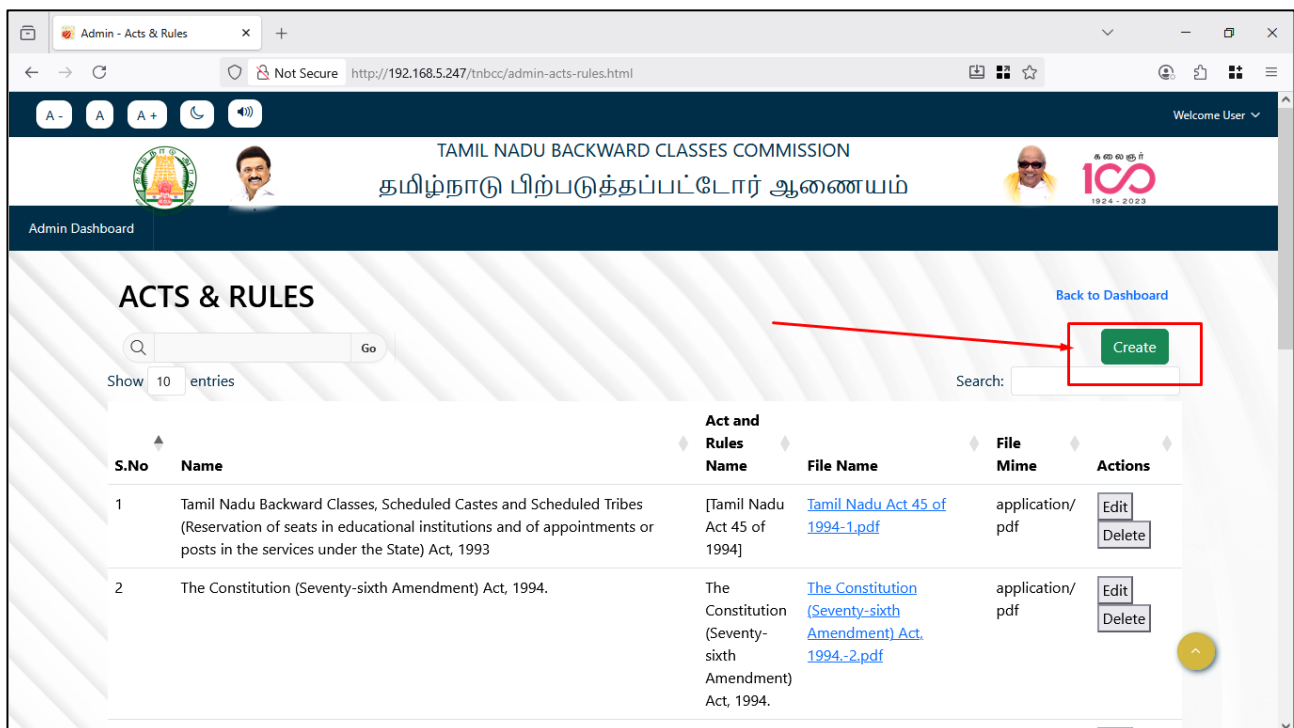
username = ADMIN

password = Admin@123

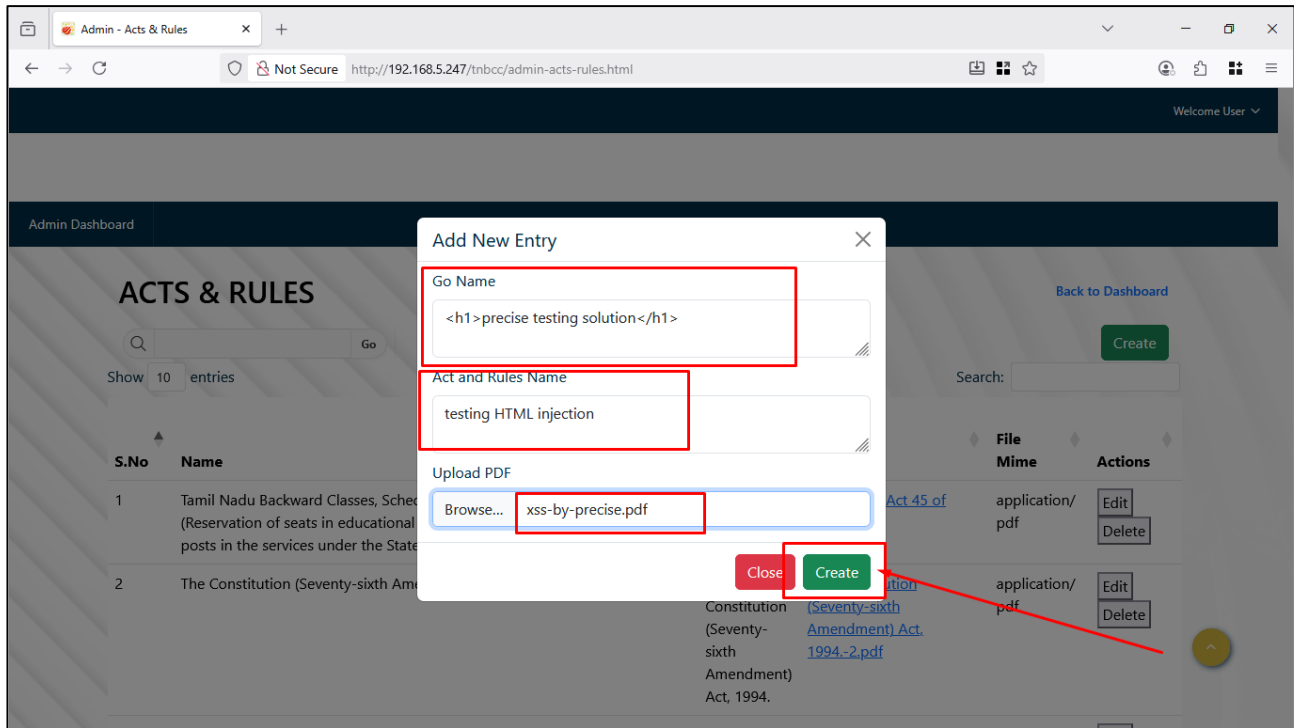
Step 3:- Click on *Acts & Rules*



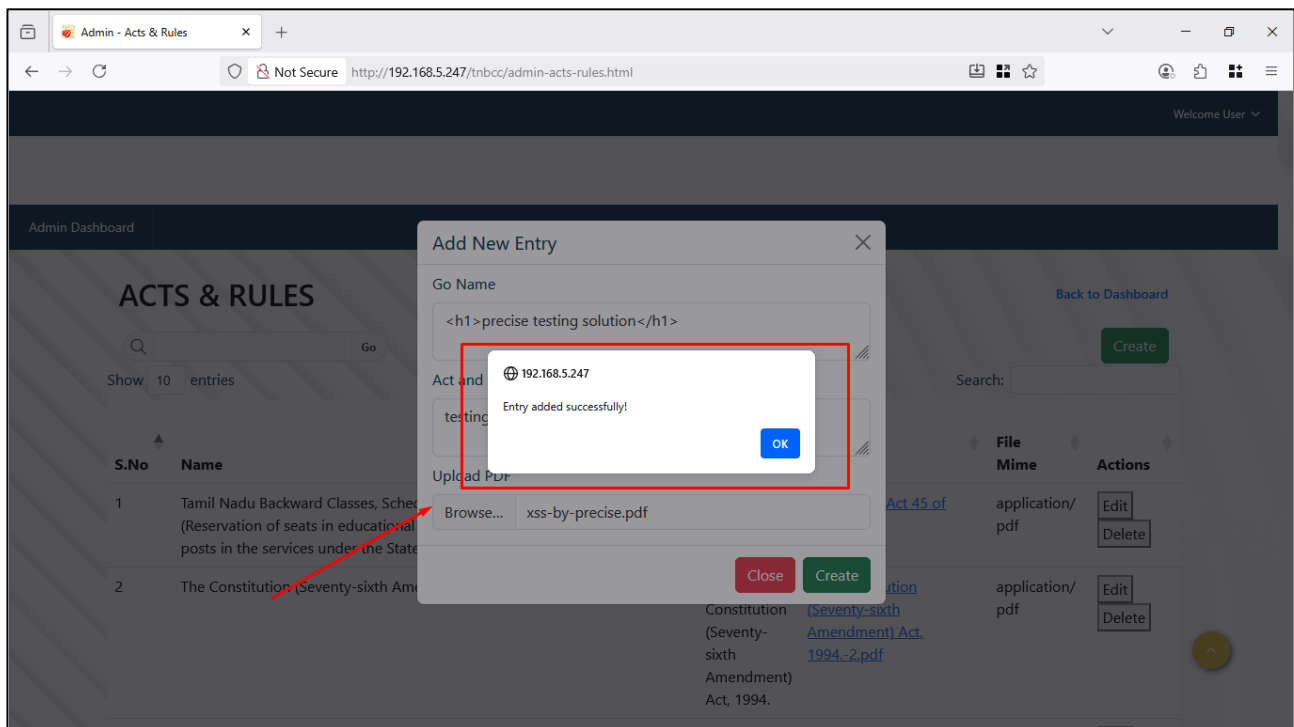
Step 4:- Click on create button



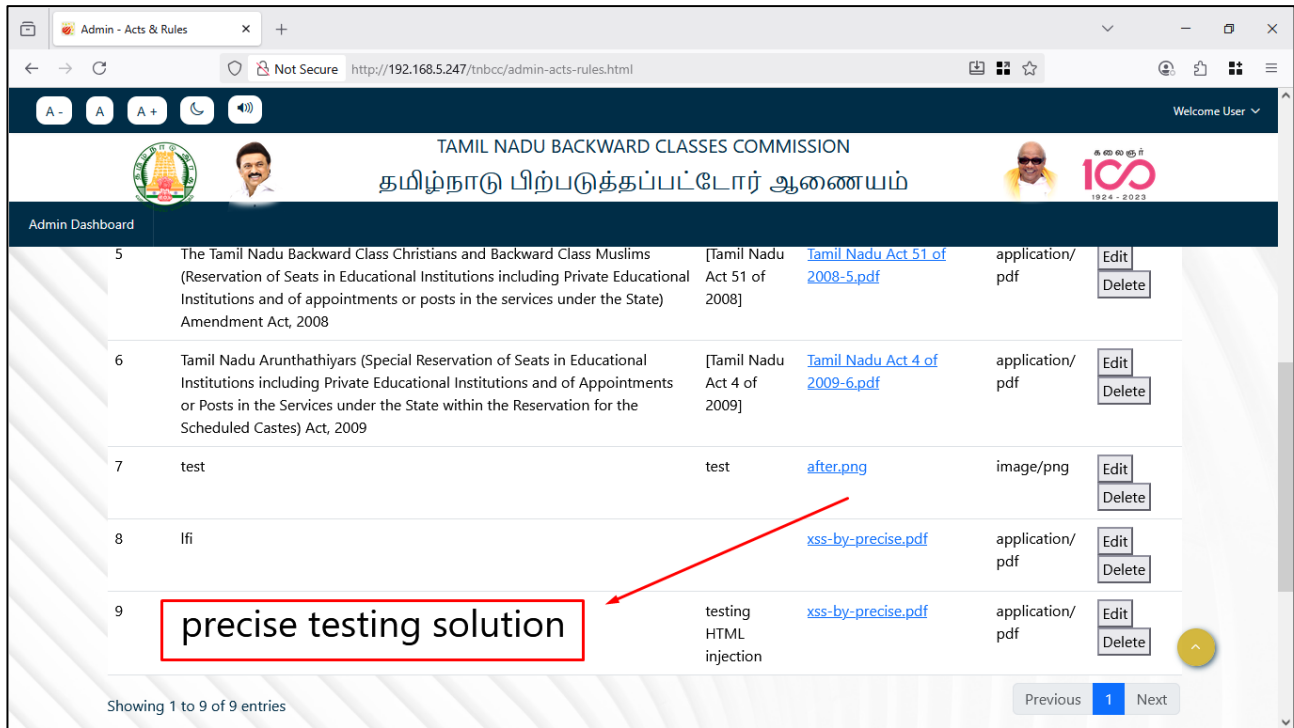
Step 5:- enter normal HTML payload <h1>precise testing solution</h1> in Go Name Field fill the form and choose any pdf file then create new entry



Step 6:- Entry added successfully!



Step 6:- HTML Code is stored successfully



Outcome:

- Web Application is Vulnerable to HTML Injection

SEVERITY: HIGH

#PT-LDS-BUG ID 03: Malicious File Upload

Description

Malicious File Upload is a critical security vulnerability that occurs when a web application allows users to upload files without implementing proper security measures to validate and restrict the types, content, and size of those files. An attacker can exploit this by uploading a malicious file (e.g., a web shell, executable, or malicious script) and then executing it to gain unauthorized access, compromise the server, or attack other users.

Reference Id(s):

OWASP TOP 10 - 2021	CVSS	CWE	WASC	Status	Severity
A01:2021-Broken Access Control	9.1	434	33	Open	High

Vulnerability Identified By:**How It Was Discovered**

☒ Manual
 ☒ Burp Suite
 ☐ Kali Linux
 ☐ SQLMap
 ☐ Other Tool

Vulnerable Points:

Affected URL / Sub-link)	Affected Link (Main)	User – Role / Impacted URL
http://192.168.5.247/tnbcc/admin-acts-rules.html	http://192.168.5.247/tnbcc/	User: ADMIN Pass: Admin@123 Role: admin
		http://192.168.5.247/tnbcc/admin-acts-rules.html

Impact

Implications / Consequences of not Fixing the Issue

Complete Server Compromise

An attacker can upload a web shell (e.g., a PHP, ASP, or JSP script) to gain remote command execution on the server. This can lead to full administrative control over the underlying server infrastructure.

Data Breach and Theft

With server access, attackers can exfiltrate sensitive data from the application's database, including personally identifiable information (PII), financial records, and intellectual property.

Website Defacement

Attackers can overwrite existing website files or upload new pages to deface the application, damaging the organization's reputation and credibility.

Malware Distribution Hub

The compromised server can be used to host and distribute malware, phishing kits, or other malicious files to unsuspecting visitors, making your infrastructure part of an attack campaign.

Denial-of-Service (DoS)

Attackers can upload extremely large files to consume all available disk space, leading to a denial-of-service condition where the application can no longer function.

Attack Chaining

The initial access provided by a malicious file upload can be used as a foothold to launch further attacks against internal network systems that are not directly exposed to the internet.

Reputation Damage and Legal Repercussions

A breach resulting from this vulnerability can lead to a massive loss of user trust, regulatory fines under laws like GDPR or CCPA, and costly legal battles.

Workaround:**Solution /Mitigation / Remediation**

Implement Strong File Type Validation

Use Allow Lists: Maintain a strict allow list of permitted file extensions (e.g., .jpg, .png, .pdf) and MIME types. Never use a block list, as it is easily bypassed.

Validate MIME Type: Check the Content-Type header from the request, but also validate the file's magic numbers (file signature) on the server-side to prevent spoofing.

Sanitize File Names and Content

Rename Files: Remove special characters and generate a new, random filename for the stored file (e.g., a3Fg7d.jpg). This prevents path traversal and overwrite attacks.

Scan for Malware: Integrate antivirus/anti-malware scanning for all uploaded files.

Sanitize File Content: For certain file types (e.g., SVG, XML), sanitize the content to remove potentially harmful elements and scripts.

Isolate Uploaded Files

Store Outside Webroot: Store uploaded files in a separate directory outside the web server's root. Serve them via a dedicated script or endpoint that delivers them safely.

Use a Content Delivery Network (CDN): Offload file hosting to a secure CDN, which can provide additional security layers.

Apply Strict Server-Side Configuration

Disable Execution: Configure the server to not execute scripts in the upload directory. This can be done via .htaccess rules on Apache (e.g., RemoveHandler .php .php3 .phtml) or permissions on Nginx.

Set File Size Limits: Enforce reasonable maximum file size limits to prevent DoS attacks via large uploads.

Implement Additional Security Controls

Authentication and Authorization: Ensure that file upload and subsequent access are protected by proper authentication and authorization checks.

Logging and Monitoring: Log all file upload activities, including user ID, filename, IP address, and timestamp. Monitor for suspicious upload attempts.

References Link or URL if Any:

- N/a

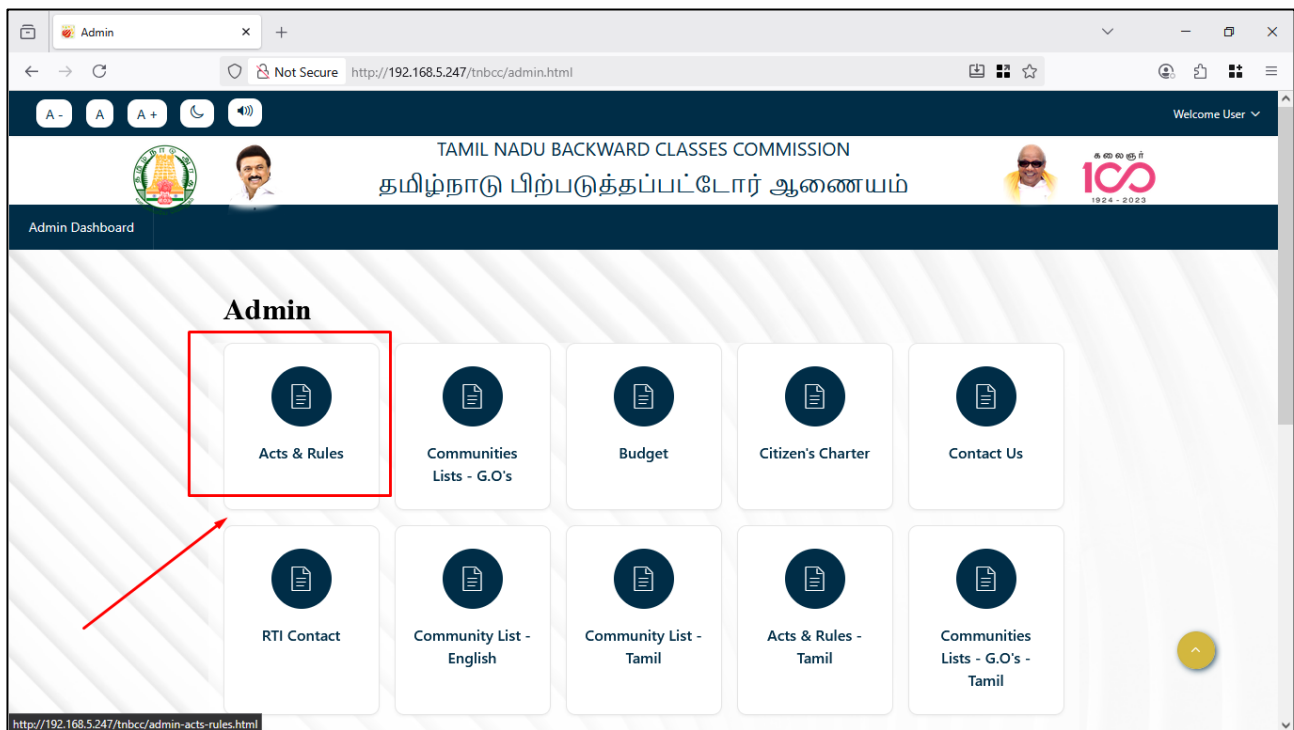
Proof Of Concept (POC)

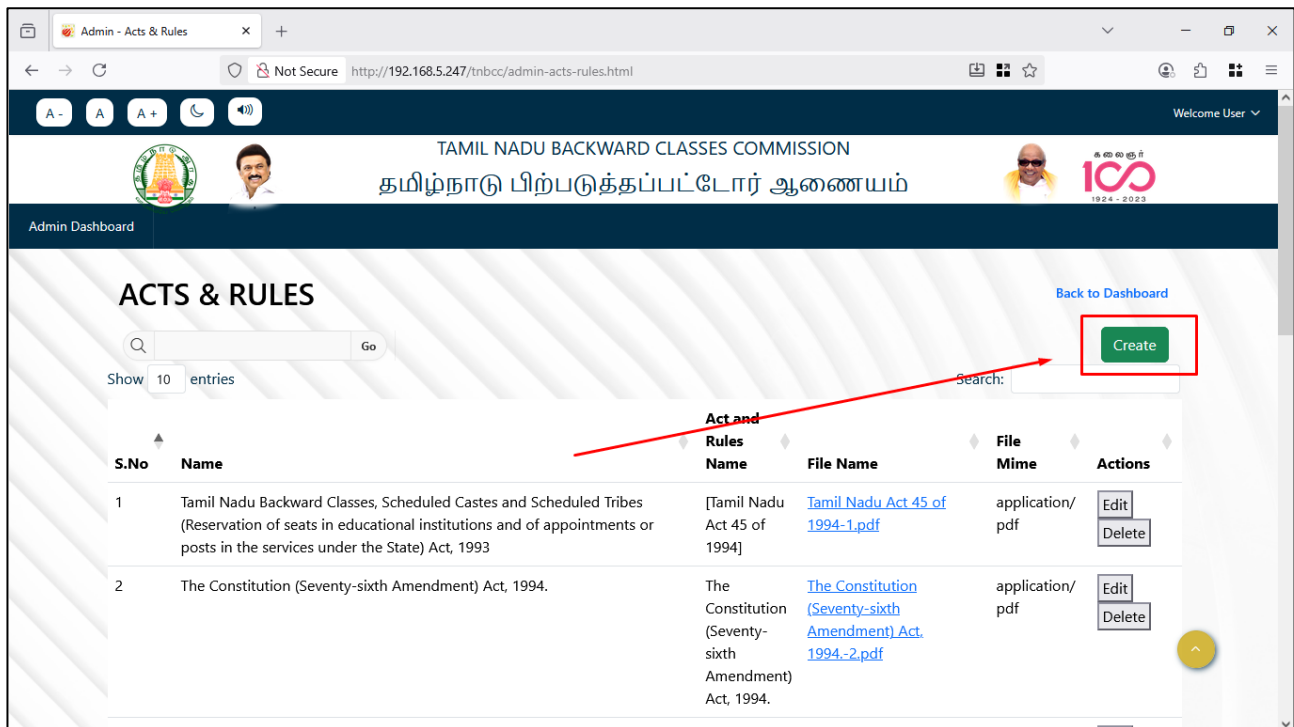
Step 1:- Login with given credentials

username = ADMIN

password = Admin@123

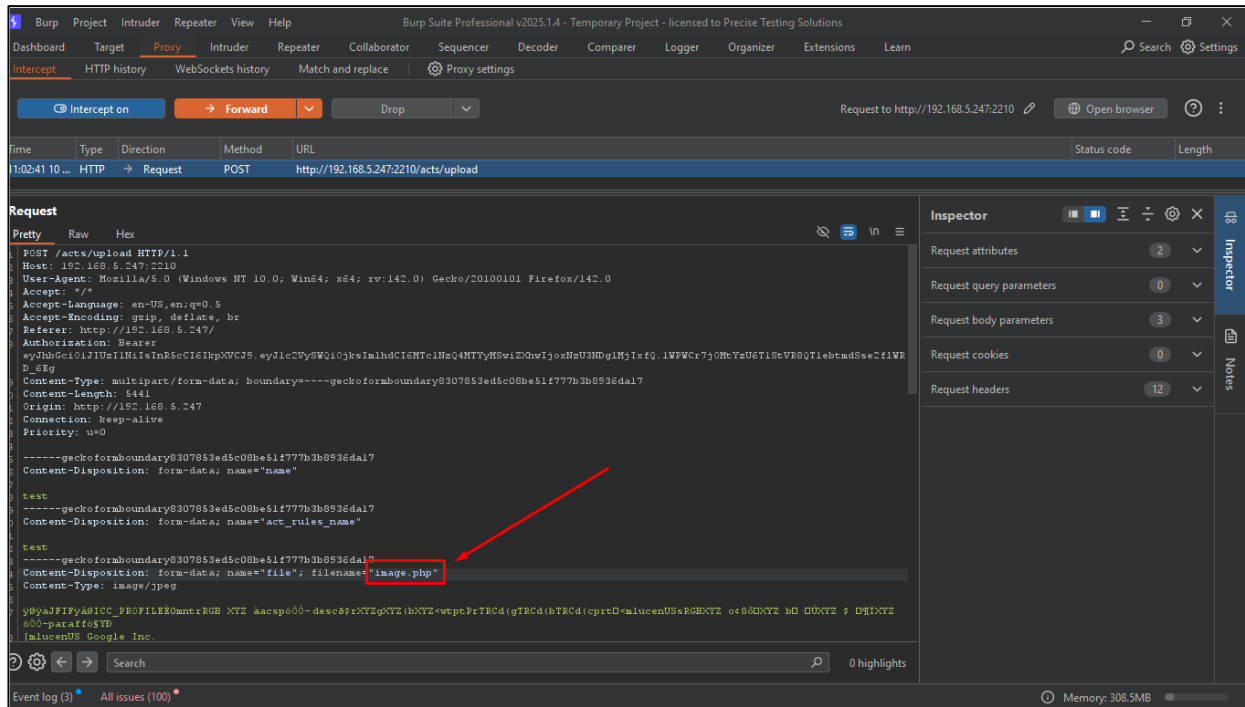
Step 2:- Click on Acts & Rules



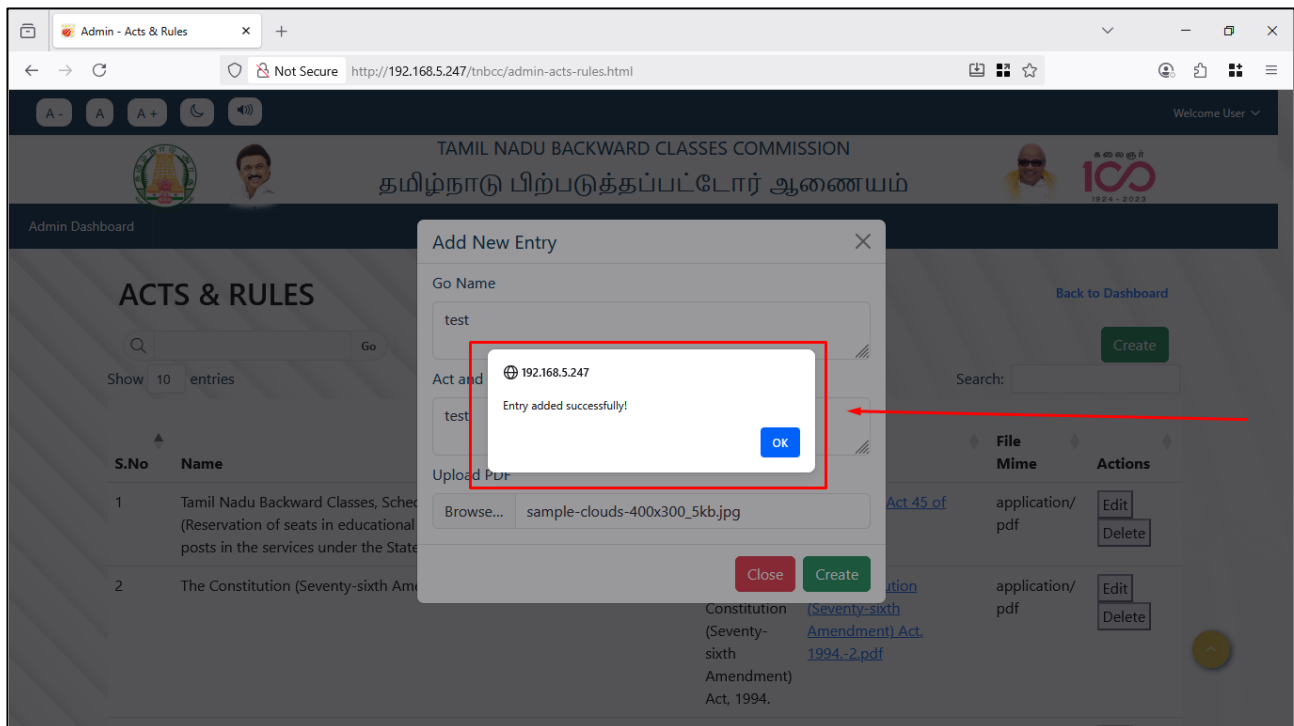
Step 3:- Click on create button**Step 4:- fill the form and choose any file then click on create new entry**

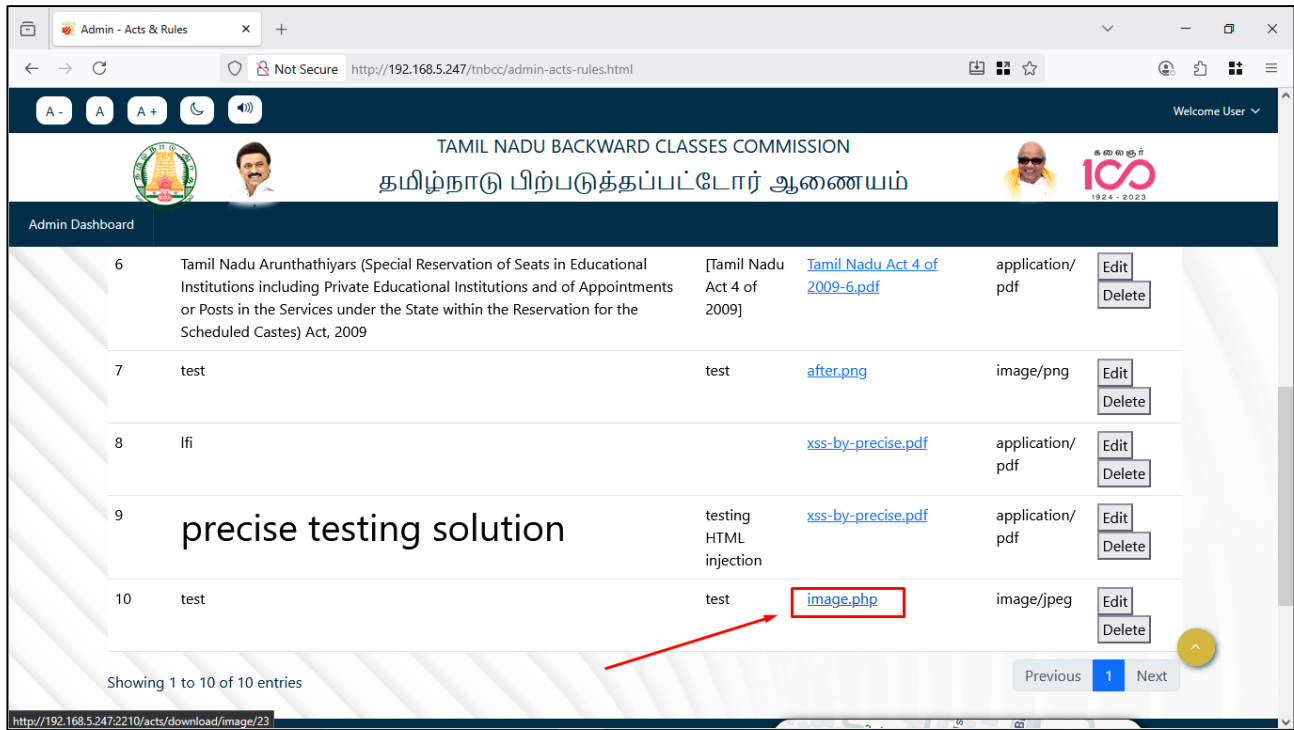
- capture the request from burp-suite

Step 5:- change the extension jpg to php and forward the request



Step 6:- Entry added successfully!





Outcome:

- Web Application is Vulnerable to ___Malicious File Upload___ Attack

SEVERITY: Medium**#PT-LDS-BUG ID 04: Browser Cache Weakness****Description**

After a user logs out or performs a sensitive action, the application fails to properly manage browser cache and session. This allows users to click the browser's "Back" button and see previously accessed sensitive pages (such as dashboards, user data, or admin panels) without proper re-authentication. This issue generally occurs when sensitive pages are cached by the browser and/or when the application does not enforce session validation on history navigation.

Reference Id(s):

OWASP TOP 10 - 2021	CVSS	CWE	Status	Severity
A01:2021-Broken Access Control	4.3	CWE: 525	Open	Medium

Vulnerability Identified By:**How It Was Discovered**

☒ Manual
 ☒ Burp Suite
 ☐ Kali Linux
 ☐ SQLMap
 ☐ Other Tool

Vulnerable Points:

Affected URL / Sub-link)	Affected Link (Main)	User – Role / Impacted URL
http://192.168.5.247/tnbcc/login.html	http://192.168.5.247/tnbcc/admin-government-rders.html	User: ADMIN Role: Admin@123
		http://192.168.5.247/tnbcc/login.html

Impact**Implications / Consequences of not Fixing the Issue**

- **Sensitive Data Exposure:** Malicious users or anyone with access to the device/browser can view sensitive data by navigating back, even after a session is supposed to have ended.
- **Potential Account Hijacking:** In shared or public computers, another user could access a previous user's sensitive data.

Workaround:**Solution /Mitigation / Remediation**

Set **no-store, no-cache, must-revalidate** cache-control headers on all sensitive pages:

Always validate the session on every page load, even when accessed via browser history.

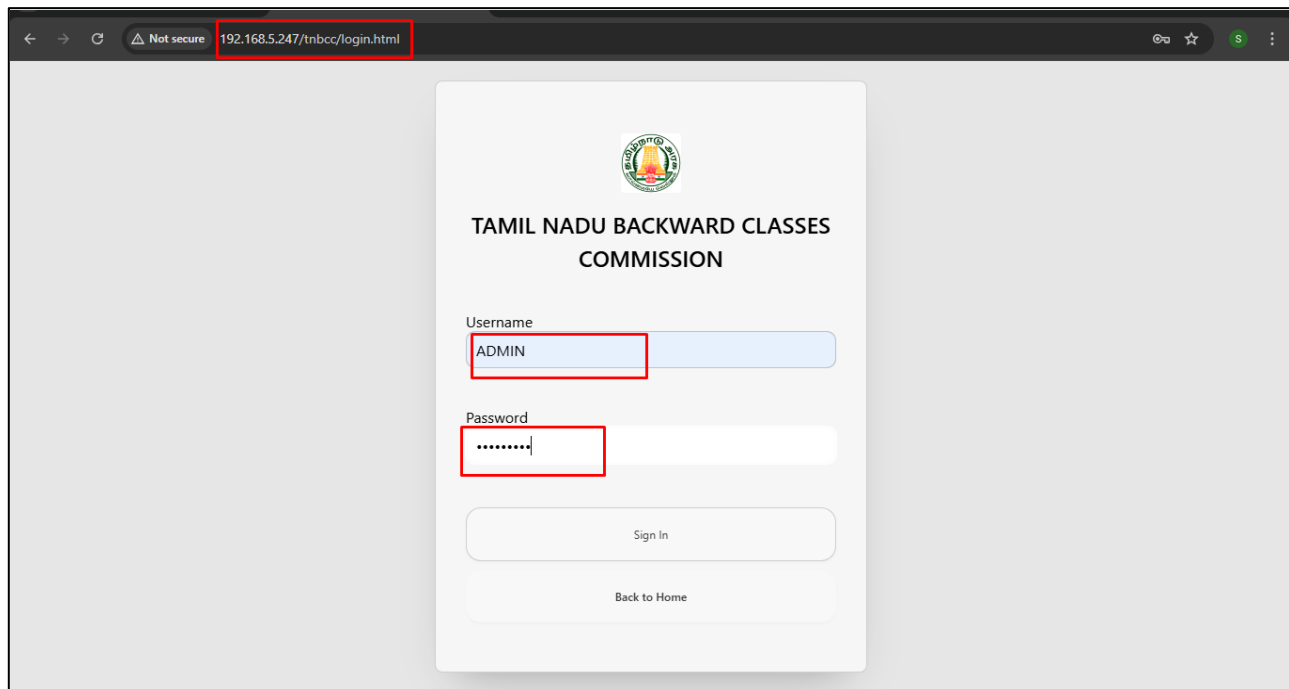
- After logout, redirect users to a neutral page and invalidate the session token server-side.
- Regularly review session and cache policies for all sensitive information and admin functionality.

References Link or URL if Any:

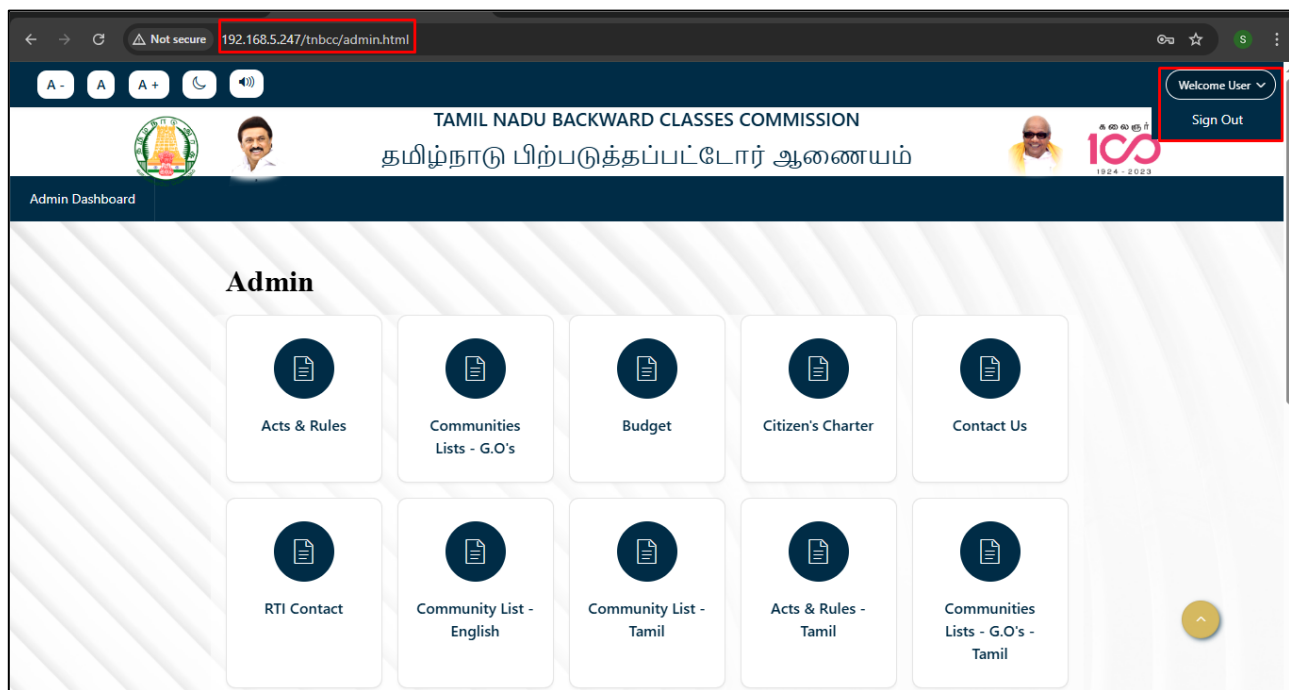
N/a

Proof Of Concept (POC)

Step 1:- Open the following link in the browser (<http://192.168.5.247/tnbcc/login.html>) and input the Credentials



Step 2:-See Open the website successfully and see we are ADMIN user



Step 3:- check the page or any in our side we are here <http://192.168.5.247/tnbcc/admin-government-rders.html>

Now click to SignOut

The screenshot shows the Admin Dashboard of the Tamil Nadu Backward Classes Commission. The page title is "TAMIL NADU BACKWARD CLASSES COMMISSION" with the Tamil text "தமிழ்நாடு பிற்படுத்தப்பட்டோர் ஆணையம்". The dashboard includes a "Government Orders" section with a table of orders. The "Sign Out" button in the top right corner is highlighted with a red box.

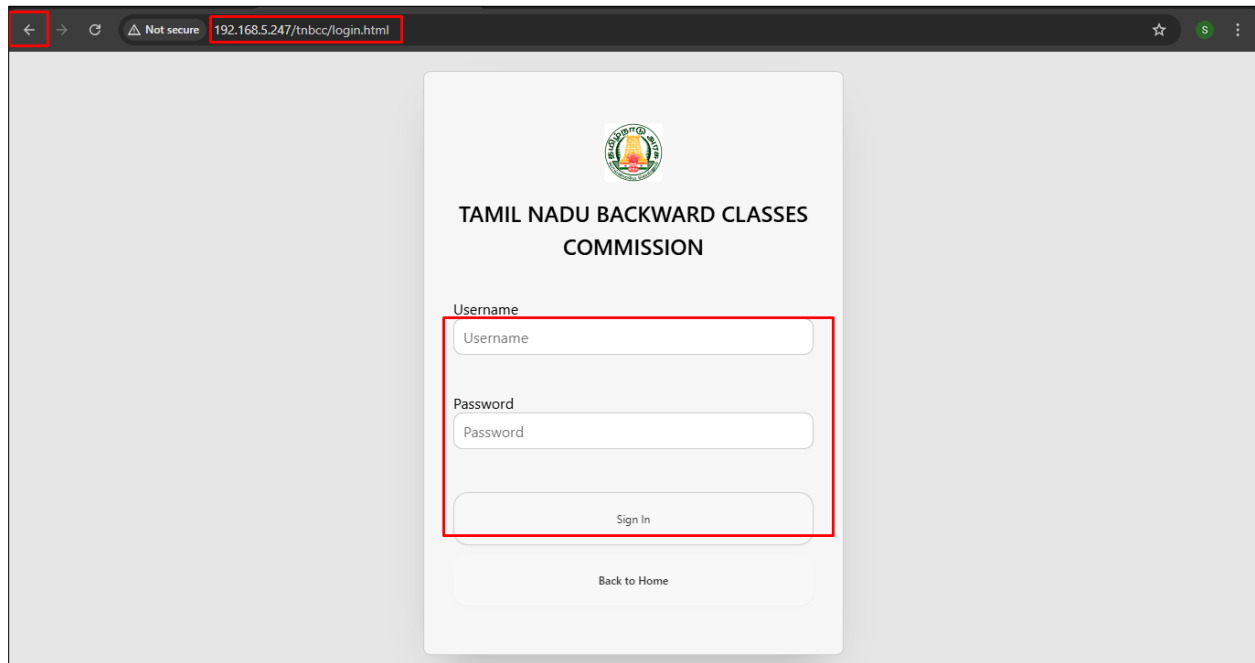
S.No	Budget Name	File Name	File Type	Actions
1	List of Backward Classes, Most Backward Classes and Denotified Communities	G.O.Ms.No.85, BC, MBC & MW (BCC) Department, dated 29.07.2008-1.pdf	application/pdf	Edit Delete
2	Acceptance of report justifying 69% reservation under the Tamil Nadu Act 45 of 1994.	G.O.Ms.No.50, BC, MBC & MW (BCC) Department, dated 11.7.2011-7.pdf	application/pdf	Edit Delete
3	Inclusion of Thiyya in BC list.	G.O.Ms.No.55, BC, MBC & MW (BCC) Department, dated 15.7.2020-9.pdf	application/pdf	Edit Delete
4	Amendment to the district names in	G.O.Ms.No.68, BC, MBC & MW (BCC) Department, dated 15.7.2020-9.pdf	application/pdf	Edit

Step 4: Click OK for SignOut to website

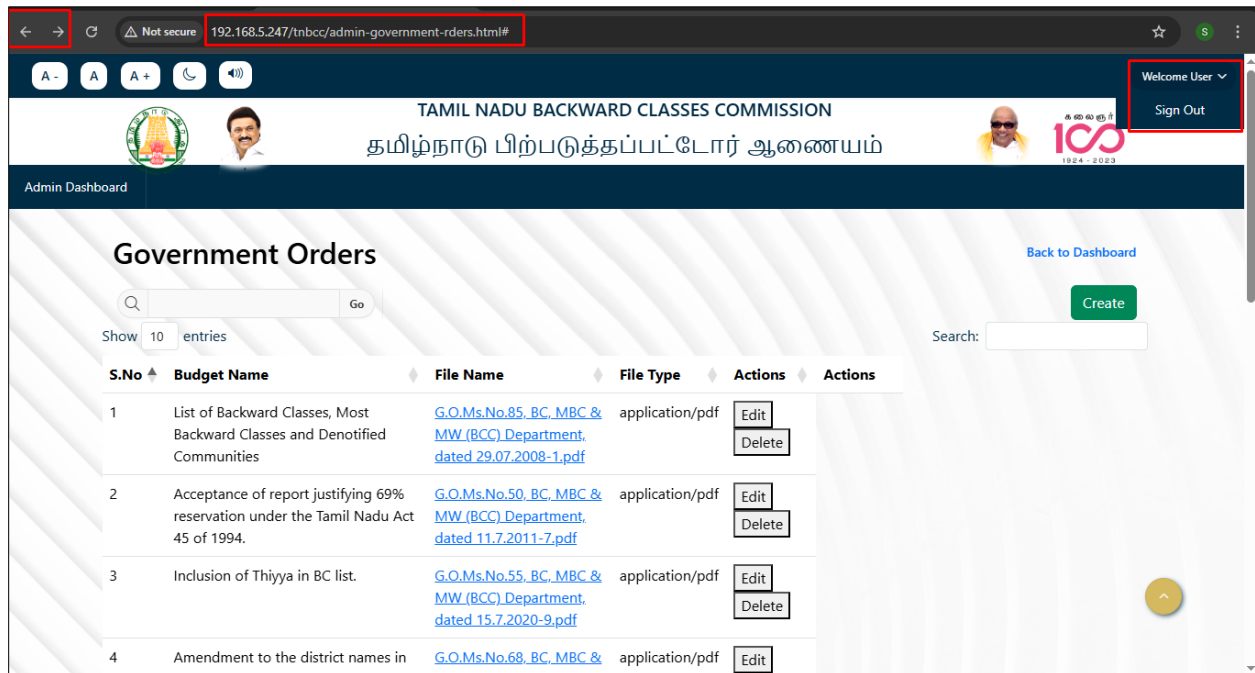
The screenshot shows the Admin Dashboard of the Tamil Nadu Backward Classes Commission. A dialog box is displayed with the text "192.168.5.247 says Signing out..." and an "OK" button. The "Sign Out" button in the top right corner is highlighted with a red box.

S.No	Budget Name	File Name	File Type	Actions
1	List of Backward Classes, Most Backward Classes and Denotified Communities	G.O.Ms.No.85, BC, MBC & MW (BCC) Department, dated 29.07.2008-1.pdf	application/pdf	Edit Delete
2	Acceptance of report justifying 69% reservation under the Tamil Nadu Act 45 of 1994.	G.O.Ms.No.50, BC, MBC & MW (BCC) Department, dated 11.7.2011-7.pdf	application/pdf	Edit Delete
3	Inclusion of Thiyya in BC list.	G.O.Ms.No.55, BC, MBC & MW (BCC) Department, dated 15.7.2020-9.pdf	application/pdf	Edit Delete
4	Amendment to the district names in	G.O.Ms.No.68, BC, MBC & MW (BCC) Department, dated 15.7.2020-9.pdf	application/pdf	Edit

Step 5: See we are here in Sign In page so now click to back button



Step 6: See Successfully Redirect / Access to website and We are in ADMIN accessed



Outcome:

Web Application is Vulnerable to Broken-Access-Control

#PT-LDS-BUG ID 05: Outdated JQuery

Description

Using an outdated version of jQuery can expose web applications to security vulnerabilities and bugs that have been addressed in more recent releases. jQuery, a widely used JavaScript library, facilitates HTML document manipulation, event handling, and animation. However, older versions may have known vulnerabilities that can be exploited by attackers.

Reference Id(s):

OWASP TOP 10 - 2021	CVSS	CWE	WASC	Status	Severity
A06:2021-Vulnerable and Outdated Components	CVSS Score 4.0 to 7.0	CWE-310	WASC-10	Open	Low

Vulnerability Identified By:**How It Was Discovered**

☒ Manual
 ☒ Burp Suite
 ☐ Kali Linux
 ☐ SQLMap
 ☐ Other Tool

Vulnerable Points:

Affected URL / Sub-link)	Affected Link (Main)	User – Role / Impacted URL
http://192.168.5.247/tnbcc/	http://192.168.5.247/tnbcc/	User: ADMIN Role: Admin@123
		http://192.168.5.247/tnbcc/

Impact**Implications / Consequences of not Fixing the Issue**

Security Vulnerabilities: Older versions of jQuery may have vulnerabilities such as:

Cross-Site Scripting (XSS): Inadequate input validation could allow malicious scripts to run.

Denial of Service (DoS): Certain exploits can lead to application crashes or unresponsive states.

Compatibility Issues: Newer browsers and features may not work correctly with outdated jQuery versions, leading to user experience problems.

Workaround:**Solution /Mitigation / Remediation**

Upgrade jQuery: Regularly update to the latest stable version of jQuery to benefit from security patches and improvements.

Use Subresource Integrity (SRI): If using jQuery from a CDN, implement SRI to ensure the integrity of the files being loaded.

Conduct Security Testing: Perform regular security assessments and vulnerability scans on your web applications.

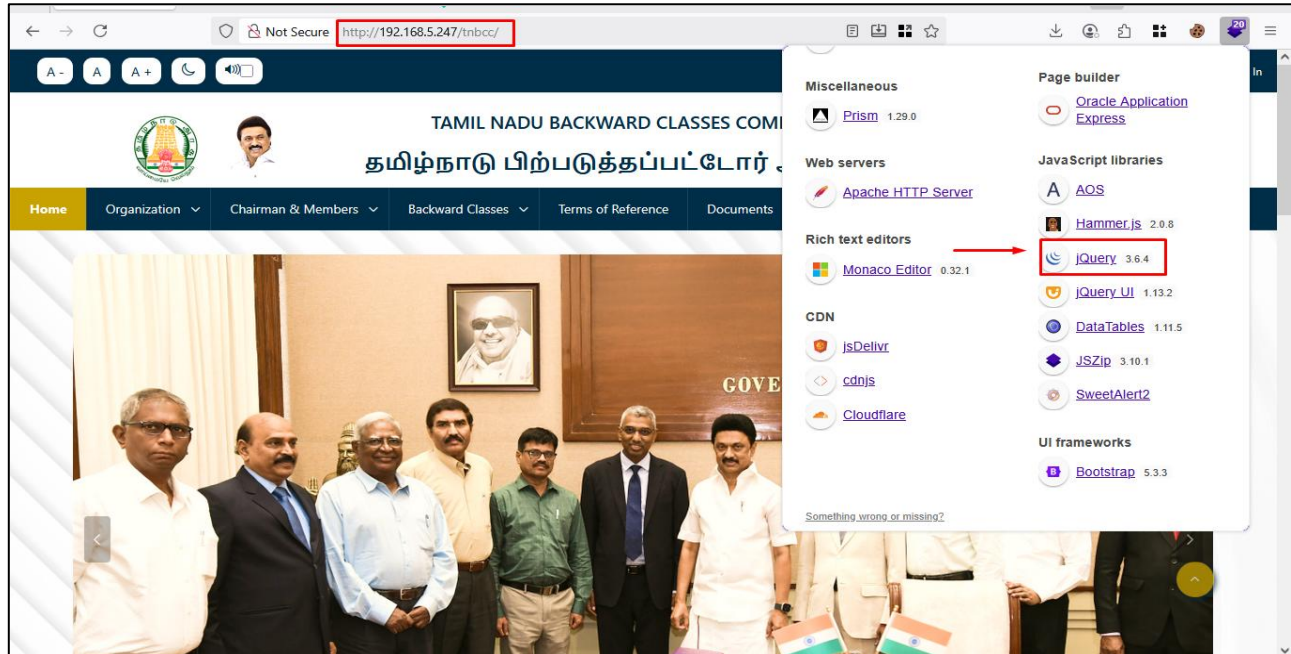
References Link or URL if Any:

N/a

Proof Of Concept (POC)

Step 1:-Open the following link and use wappalyzer tool see JQuery version outdated

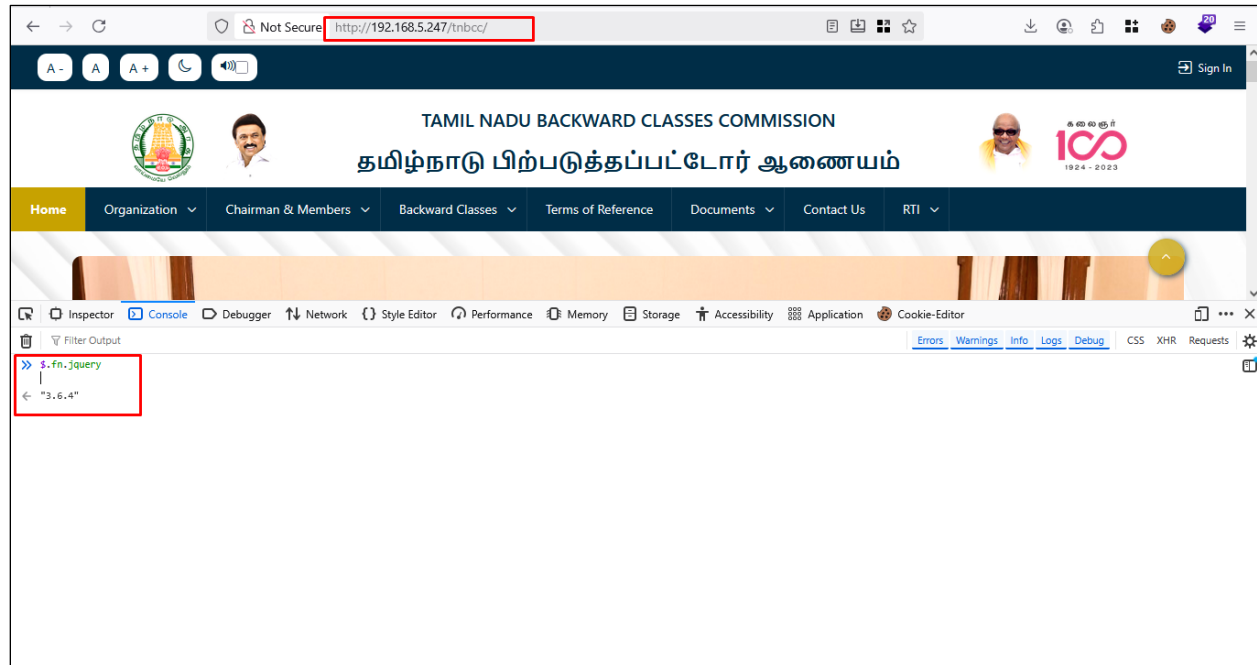
<http://192.168.5.247/tnbcc/>



Step 2: Use again in for check version Press F12 for open developer mode enable

open console and type command fn.jquery

See show outdated version



Outcome:

Web Application is Vulnerable to outdated jquery version

#PT-LDS-BUG ID 06: Outdated Bootstrap Version

Description

Using an outdated version of Bootstrap, a popular front-end framework, can lead to security vulnerabilities, compatibility issues, and performance drawbacks. Bootstrap provides pre-built components and styles for web development, and older versions may contain known security flaws that can be exploited by attackers.

Reference Id(s):

OWASP TOP 10 - 2021	CVSS	CWE	WASC	Status	Severity
A06:2021-Vulnerable and Outdated Components	CVSS: 4.0 to 6.0	CWE: 1021, 346	WASC- 10	Open	Low

Vulnerability Identified By:

How It Was Discovered

☒ Manual
 ☐ Burp Suite
 ☐ Kali Linux
 ☐ SQLMap
 ☐ Other Tool

Vulnerable Points:

Affected URL / Sub-link)	Affected Link (Main)	User – Role / Impacted URL
http://192.168.5.247/tnbcc/	http://192.168.5.247/tnbcc/	User: ADMIN Role: Admin@123
		http://192.168.5.247/tnbcc/

Impact

Implications / Consequences of not Fixing the Issue

Security Vulnerabilities: Older versions of Bootstrap may have vulnerabilities, such as:

Cross-Site Scripting (XSS): Inadequate handling of user inputs can allow malicious scripts to execute.

CSS and JavaScript Bugs: Known issues may lead to unintended behavior or security risks in the user interface.

Compatibility Issues: Newer browsers or features may not work optimally with outdated Bootstrap versions, potentially affecting user experience.

Workaround:

Solution /Mitigation / Remediation

Upgrade Bootstrap: Regularly update to the latest stable version of Bootstrap to benefit from security patches, new features, and improvements.

Review Release Notes: Understand the changes and fixes made in each version to evaluate the impact of upgrading.

Conduct Security Audits: Regularly perform security assessments on your web application to identify and remediate vulnerabilities.

References Link or URL if Any:

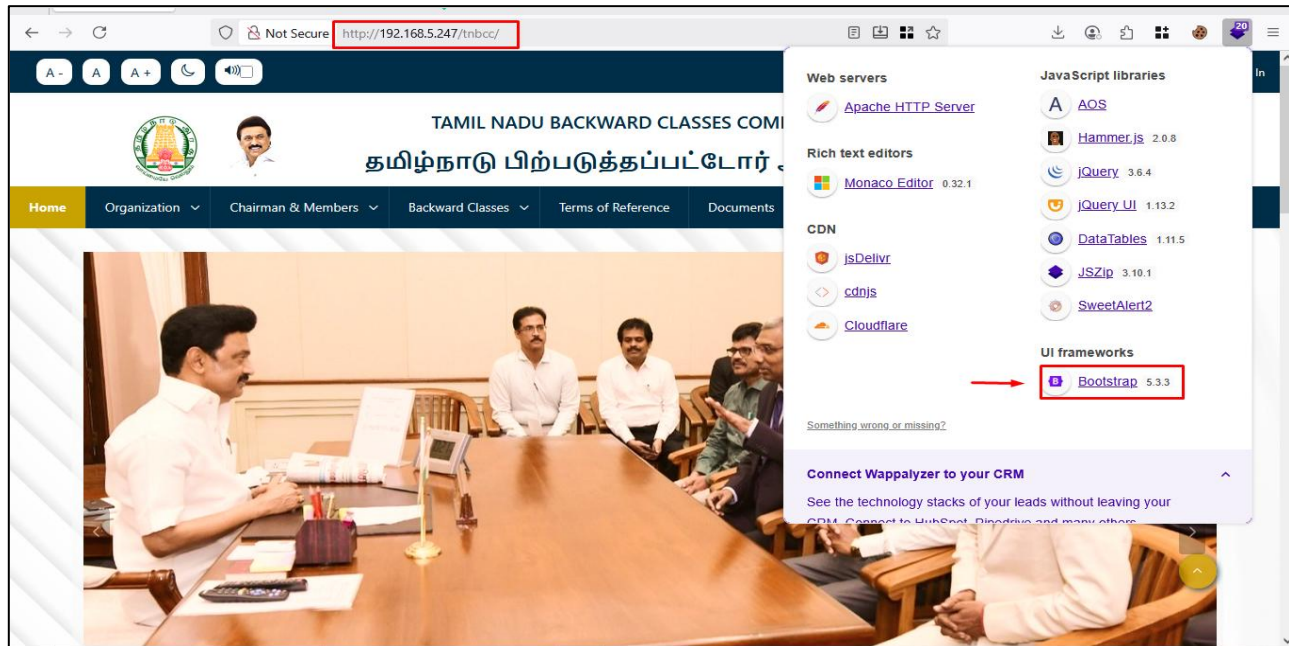
- N/A
-

Proof Of Concept (POC)

Step 1:-Open the following link as shown in POC

<http://192.168.5.247/tnbcc/>

Step 2: Use wappalyzer tool it show Outdated version of Bootstrap



Outcomes:

- Web Application is Vulnerable to outdated bootstrap version

SEVERITY: LOW**#PT-LDS-BUG ID 07: Server Name Disclosure****Description**

The HTTP responses returned by this web application include a header named **Server**. The value of this header includes the version of apache server.

Reference Id(s):

OWASP TOP 10 - 2021	CVSS	CWE	Status	Risk Rating
A05:2021-Security Misconfiguration	N/A	CWE-200	Open	Low

- How It Was Discovered

☐ Manual
 ☒ Burp Suite
 ☐ Kali Linux
 ☐ SQLMap
 ☐ Other Tool

Vulnerable Points:

Affected URL / Sub-link)	Affected Link (Main)	User – Role / Impacted URL
http://192.168.5.247/tnbcc/	http://192.168.5.247/tnbcc/	User: N/A Role: N/A
		http://192.168.5.247/tnbcc/

Impact**Implications / Consequences of not Fixing the Issue**

An attacker might use the disclosed information to harvest specific security vulnerabilities for the version identified.

Workaround:**Solution /Mitigation / Remediation**

Steps to reduce the Risk with following technique as Solution:

Microsoft IIS should be configured to remove unwanted HTTP response headers from the response.

Server: Microsoft-IIS/10.0

X-Powered-By: PHP/7.4.27

X-Powered-By: ASP.NET

References Link or URL if Any:

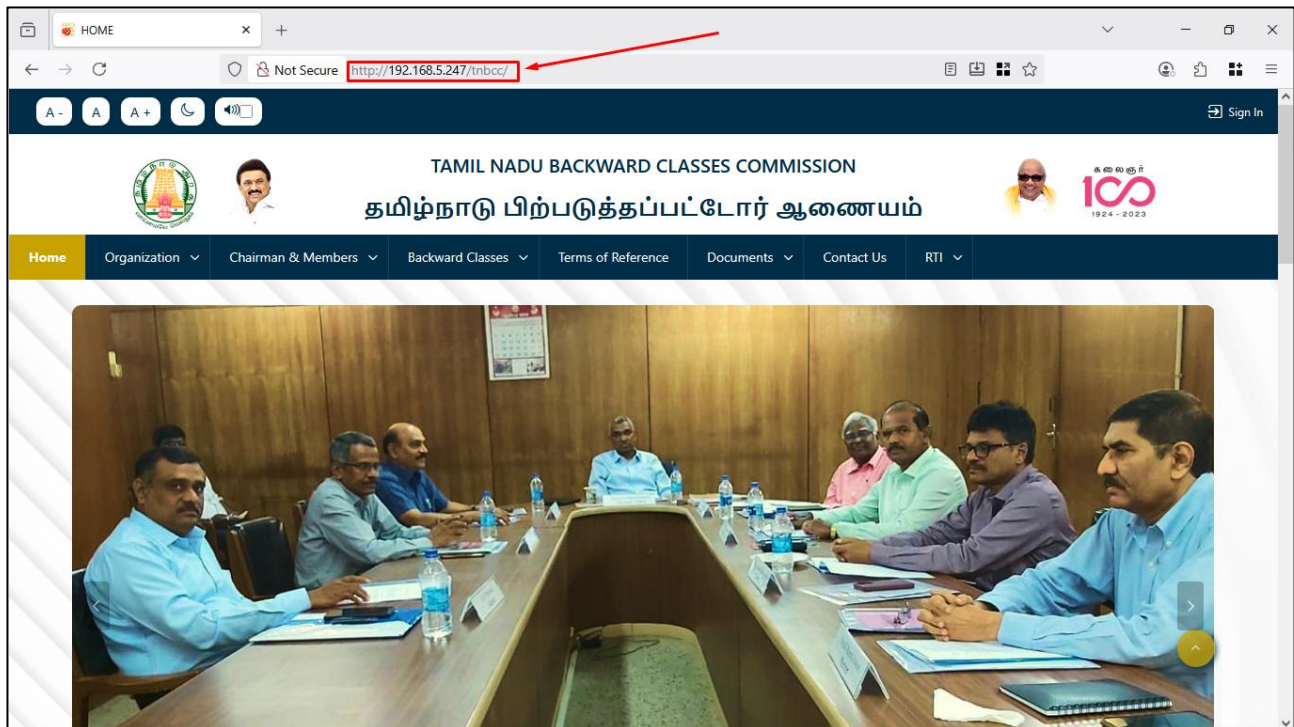
- N/a

Proof Of Concept (POC)

Step 1:-

- Open a Link in browser

<http://192.168.5.247/tnbcc/>



Step 2:-

- Intercept the request through Burp Suite
- Send intercepted request to repeater
- Click on send button
- **Server name disclose in Response**

The screenshot displays the Burp Suite interface with the 'Repeater' tab selected. The 'Request' pane on the left shows an intercepted GET request to `/cnbce/ HTTP/1.1` from host `192.168.5.247`. The 'Response' pane on the right shows the corresponding HTTP response. A red arrow points to the `Server: Apache` header in the response, indicating a server name disclosure. The 'Inspector' pane on the right shows the request and response details.

Request:

```

1 GET /cnbce/ HTTP/1.1
2 Host: 192.168.5.247
3 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:142.0)
  Gecko/20100101 Firefox/142.0
4 Accept:
  text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Connection: Keep-Alive
8 Upgrade-Insecure-Requests: 1
9 Priority: u=0, i
10
11

```

Response:

```

1 HTTP/1.1 200 OK
2 Date: Wed, 10 Sep 2025 05:38:05 GMT
3 Server: Apache
4 Access-Control-Allow-Methods: GET, POST
5 Access-Control-Allow-Headers: Content-Type, Authorization
6 Access-Control-Allow-Credentials: true
7 Strict-Transport-Security: max-age=31536000; includeSubDomains;
  preload
8 X-Frame-Options: SAMEORIGIN
9 Last-Modified: Sat, 09 Nov 2024 09:05:46 GMT
10 ETag: "f1a1-626772c45ce80-gzip"
11 Accept-Ranges: bytes
12 Vary: Accept-Encoding
13 Access-Control-Allow-Origin: 192.168.5.247, tngis.tnaga.org
14 Permissions-Policy: geolocation=(self), microphone=(),
  camera=(), fullscreen=*, payment=(), accelerometer=(),
  autoplay=(), encrypted-media=(), midi=(), sync-xhr=(), usb=()
15 Referrer-Policy: strict-origin-when-cross-origin
16 Content-Security-Policy: default-src *; script-src *
  'unsafe-inline' 'unsafe-eval'; style-src * 'unsafe-inline';
  img-src * data: 'unsafe-inline'; connect-src * 'unsafe-inline';
17 X-XSS-Protection: 1; mode=block
18 X-Content-Type-Options: nosniff
19 X-Permitted-Cross-Domain-Policies: none
20 Cache-Control: no-store, no-cache, must-revalidate, max-age=0
21 Pragma: no-cache
22 Expires: 0
23 Content-Length: 61857
24 Keep-Alive: timeout=5, max=100
25 Connection: Keep-Alive
26 Content-Type: text/html
27
28 <!DOCTYPE html>

```

Outcome:

Web application is vulnerable to **Server Name Disclosure**

THANK YOU!



PRECISE
Testing Solution



- : End Of Report:-

- : End Of Report:-